

Threads in Python

Quick Review - Part 2





The main idea of this class is a hand-on a section to learning programming **Thread** and **Multi-Processing** with Python in a practical fashion.

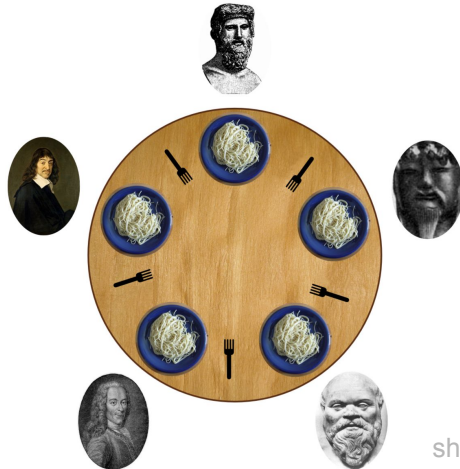


Topics

- What is concurrency?
- Philosophers
 - ◆ RLocks
- Locks
- Conditionals

What is concurrency

Concurrency is, essentially, the practice of doing multiple things at the same time, but not, specifically, in parallel. It can help us to improve the perceived performance of our applications, and it can also improve the speed at which our applications run.



shorturl.at/lqxVZ

Philosophers

```
1
2 import threading
3 import time
4 import random
5
6 class Philosopher(threading.Thread):...
33
34     fork1 = threading.RLock()
35     fork2 = threading.RLock()
36     fork3 = threading.RLock()
37     fork4 = threading.RLock()
38     fork5 = threading.RLock()
39
40     philosopher1 = Philosopher("Kant", fork1, fork2)
41     philosopher2 = Philosopher("Aristotle", fork2, fork3)
42     philosopher3 = Philosopher("Spinoza", fork3, fork4)
43     philosopher4 = Philosopher("Marx", fork4, fork5)
44     philosopher5 = Philosopher("Russell", fork5, fork1)
45
```

Philosophers

46	<code>philosopher1.start()</code>
47	<code>philosopher2.start()</code>
48	<code>philosopher3.start()</code>
49	<code>philosopher4.start()</code>
50	<code>philosopher5.start()</code>
51	
52	<code>philosopher1.join()</code>
53	<code>philosopher2.join()</code>
54	<code>philosopher3.join()</code>
55	<code>philosopher4.join()</code>
56	<code>philosopher5.join()</code>

Philosophers

```
6 class Philosopher(threading.Thread):
7     def __init__(self, name, leftFork, rightFork):...
12
13     def run(self):...
37
```

Philosophers

```
6  class Philosopher(threading.Thread):
7      def __init__(self, name, leftFork, rightFork):
8          print("{} Has Sat Down At the Table".format(name))
9          threading.Thread.__init__(self, name=name)
10         self.leftFork = leftFork
11         self.rightFork = rightFork
```

```
13 def run(self):
14     while True:
15         print("{} has started thinking".format(
16             threading.currentThread().getName()))
17         time.sleep(random.randint(1, 5))
18         print("{} has finished thinking".format(
19             threading.currentThread().getName()))
20         self.leftFork.acquire()
21         time.sleep(random.randint(1, 5))
22         try:
23             print("{} has acquired the left fork".format(
24                 threading.currentThread().getName()))
25             self.rightFork.acquire()
26             try:
27                 print("{} has attained both forks, currently eating".format(
28                     threading.currentThread().getName()))
29             finally:
30                 self.rightFork.release()
31                 print("{} has released the right fork".format(
32                     threading.currentThread().getName()))
33             finally:
34                 self.leftFork.release()
35                 print("{} has released the left fork".format(
36                     threading.currentThread().getName()))
```

Philosophers: Output

```
F1 Has Sat Down At the Table
F2 Has Sat Down At the Table
F3 Has Sat Down At the Table
F4 Has Sat Down At the Table
F5 Has Sat Down At the Table
F1 has started thinking
F2 has started thinking
F3 has started thinking
F4 has started thinking
F5 has started thinking
F5 has finished thinking
F1 has finished thinkingF3 has finished thinking

F3 has acquired the left forkF4 has finished thinking

F5 has acquired the left fork
F2 has finished thinking
```

Philosophers: Question

Is the current Philosophers code free of Deadlock?
Why? Who to solve this issue?

RLocks

RLock Objects: A reentrant lock is a synchronization primitive that may be acquired multiple times by the same thread.

RLocks: Simple Example

```
1  import threading
2  import time
3
4  def teste(lock):
5      lock.acquire()
6      print(threading.current_thread())
7      time.sleep(1)
8      # lock.release()
9
10 l1 = threading.RLock()
11 t1 = threading.Thread(target=teste, args=(l1,))
12 t2 = threading.Thread(target=teste, args=(l1,))
13 t1.start()
14 t1.join()
15 t2.start()
16 t2.join()
17
18 print("Hello")
```

What would happen if the comment tag on line 8 is removed?

Lock: A not reentrant lock

Lock: an object that can only be owned by a single thread at any given time. Basic operations on a lock:

- **acquire:** mark the lock as owned by the current thread; if some other thread already owns the lock then first wait until the lock is free. Lock typically includes a queue to keep track of multiple waiting threads.
- **release:** mark the lock as free (it must currently be owned by the calling thread).

RLOCK More Information

<https://docs.python.org/3/library/threading.html?highlight=rlock#threading.RLock>



Lock: A not reentrant lock

```
1  import threading
2  import time
3
4  # l1 = threading.RLock()
5  l1 = threading.Lock()
6  l1.acquire()
7  l1.acquire()
8  |
9  print("Hello")
10
```

What will happen
in this code?

Condition

A condition is a synchronization primitive that waits on a signal from another thread. For example, this could be that another thread has finished execution, and that the current thread can proceed to perform some kind of calculation on the results.

Condition

A condition is a synchronization primitive that waits on a signal from another thread. For example, this could be that another thread has finished execution, and that the current thread can proceed to perform some kind of calculation on the results.

Condition

Synchronization mechanisms need more than just mutual exclusion; also need a way to wait for another thread to do something (e.g., wait for a character to be added to the buffer)

Condition: wait()

wait(timeout=None): Wait until notified or until a timeout occurs. If the calling thread has not acquired the lock when this method is called, a `RuntimeError` is raised. **This method releases the underlying lock.**

Condition: Producer and Consumer Example

```
1  import random
2  import threading
3  import time
4
5
6  class Producer(threading.Thread):...
25
26
27  class Consumer(threading.Thread):...
46
47  def main():
48      integers = []
49      condition = threading.Condition()
50      t1 = Producer(integers, condition)
51      t2 = Consumer(integers, condition)
52      t1.start()
53      t2.start()
54      t1.join()
55      t2.join()
56
57  if __name__ == '__main__':
58      main()
```

```
1 import random
2 import threading
3 import time
4
5
6 class Producer(threading.Thread):
7     def __init__(self, integers, condition):
8         threading.Thread.__init__(self)
9         self.integers = integers
10        self.condition = condition
11
12    def run(self):
13        for x in range(10):
14            # integer = random.randint(0, 256)
15            integer = x
16            self.condition.acquire()
17            print('condition acquired by %s' % self.name)
18            self.integers.append(integer)
19            print('%d appended to list by %s' % (integer, self.name))
20            print('condition notified by %s' % self.name)
21            self.condition.notify()
22            print('condition released by %s' % self.name)
23            self.condition.release()
24            time.sleep(10)
```

Condition: Producer and Consumer Example

```
27 class Consumer(threading.Thread):
28     def __init__(self, integers, condition):
29         threading.Thread.__init__(self)
30         self.integers = integers
31         self.condition = condition
32
33     def run(self):
34         while True:
35             self.condition.acquire()
36             print('condition acquired by %s' % self.name)
37             while True:
38                 if self.integers:
39                     integer = self.integers.pop()
40                     print('%d popped from list by %s' % (integer, self.name))
41                     break
42             print('condition wait by %s' % self.name)
43             self.condition.wait()
44             print('condition released by %s' % self.name)
45             self.condition.release()
```

Condition: Producer and Consumer Example

```
47  def main():  
48      integers = []  
49      condition = threading.Condition()  
50      t1 = Producer(integers, condition)  
51      t2 = Consumer(integers, condition)  
52      t1.start()  
53      t2.start()  
54      t1.join()  
55      t2.join()  
56  
57  if __name__ == '__main__':  
58      main()  
59
```

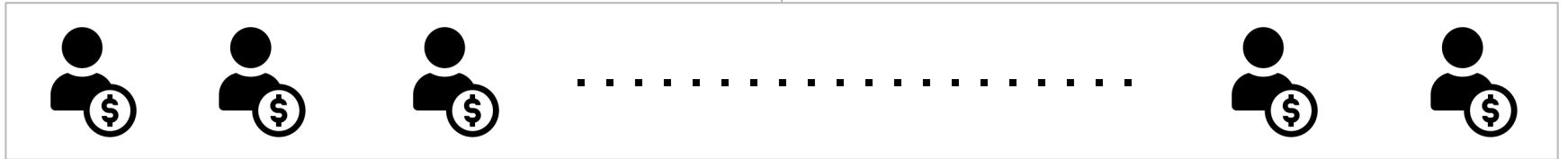
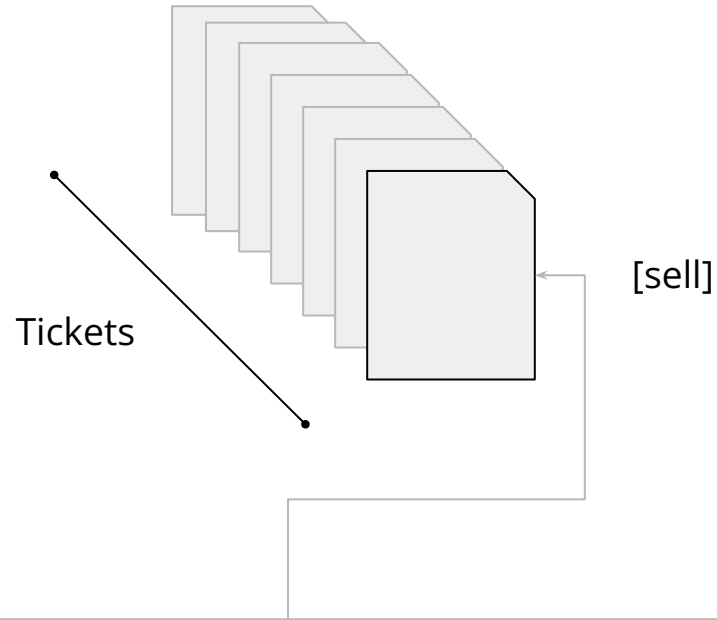
Results

```
$ python3.6 03_pubSub.py
Condition Acquired by Publisher: Thread-1
Publisher Thread-1 appending to array: 108
Condition Released by Publisher: Thread-1
Condition Acquired by Consumer: Thread-2
108 Popped from list by Consumer: Thread-2
Consumer Thread-2 Releasing Condition
Condition Acquired by Consumer: Thread-2
Condition Wait by Thread-2
Condition Acquired by Consumer: Thread-3
Condition Wait by Thread-3
Condition Acquired by Publisher: Thread-1
Publisher Thread-1 appending to array: 563
...
```

Semaphores

Semaphores have an internal counter that is incremented and decremented whenever either an acquire or a release call is made. Upon initialization, this counter defaults to 1 unless otherwise set. The semaphore cannot be acquired if the counter will fall to a negative integer value.

Example: TicketSellers



Example: TicketSellers

```
1  import threading
2  import time
3  import random
4
5  class TicketSeller(threading.Thread):...
31
32  semaphore = threading.Semaphore()
33  ticketsAvailable = 200
34
35  sellers = []
36  for i in range(4):
37      seller = TicketSeller(semaphore)
38      seller.start()
39      sellers.append(seller)
40
41  for seller in sellers:
42      seller.join()
```

Example: TicketSellers

```
6 class TicketSeller(threading.Thread):
7     ticketsSold = 0
8
9     def __init__(self, semaphore):...
13
14 def run(self):...
31
32 def randomDelay(self):
33     time.sleep(random.randint(0, 4) / 4)
34
```

Example: TicketSellers

```
6  class TicketSeller(threading.Thread):  
7      ticketsSold = 0  
8  
9      def __init__(self, semaphore):  
10         threading.Thread.__init__(self);  
11         self.sem = semaphore  
12         print("Ticket Seller Started Work")  
13
```

Example: TicketSellers

```
14 def run(self):
15     global ticketsAvailable
16     running = True
17     while running:
18         self.randomDelay()
19
20         self.sem.acquire()
21         if (ticketsAvailable <= 0):
22             running = False
23         else:
24             self.ticketsSold = self.ticketsSold + 1
25             ticketsAvailable = ticketsAvailable - 1
26             print("{} Sold One ({} left)".format(self.getName(),
27                                                    ticketsAvailable))
28             self.sem.release()
29             print("Ticket Seller {} Sold {} tickets in total".format(self.getName(),
30                                                                           self.ticketsSold))
31
```

TicketSellers: Results

```
pythonThreads/ticketSellers.py
```

```
Ticket Seller Started Work
```

```
Ticket Seller Started Work
```

```
Ticket Seller Started Work
```

```
Ticket Seller Started Work
```

```
Thread-1 Sold One (5 left)
```

```
Thread-1 Sold One (4 left)
```

```
Thread-2 Sold One (3 left)
```

```
Thread-2 Sold One (2 left)
```

```
Thread-3 Sold One (1 left)
```

```
Thread-4 Sold One (0 left)
```

```
Ticket Seller Thread-1 Sold 2 tickets in total
```

```
Ticket Seller Thread-4 Sold 1 tickets in total
```

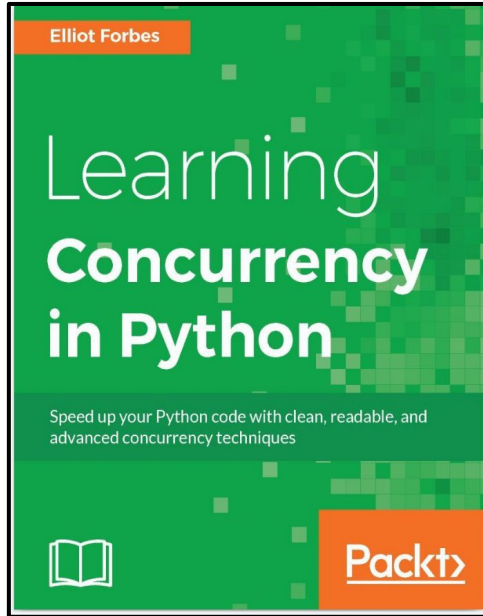
```
Ticket Seller Thread-3 Sold 1 tickets in total
```

```
Ticket Seller Thread-2 Sold 2 tickets in total
```

**With only 6
tickets
available.**

```
36 semaphore = threading.Semaphore()  
37 ticketsAvailable = 6
```

Reference (this class is based on this book)



shorturl.at/abhqL